



SISTEMAS OPERATIVOS DE TIEMPO REAL(RTOS)

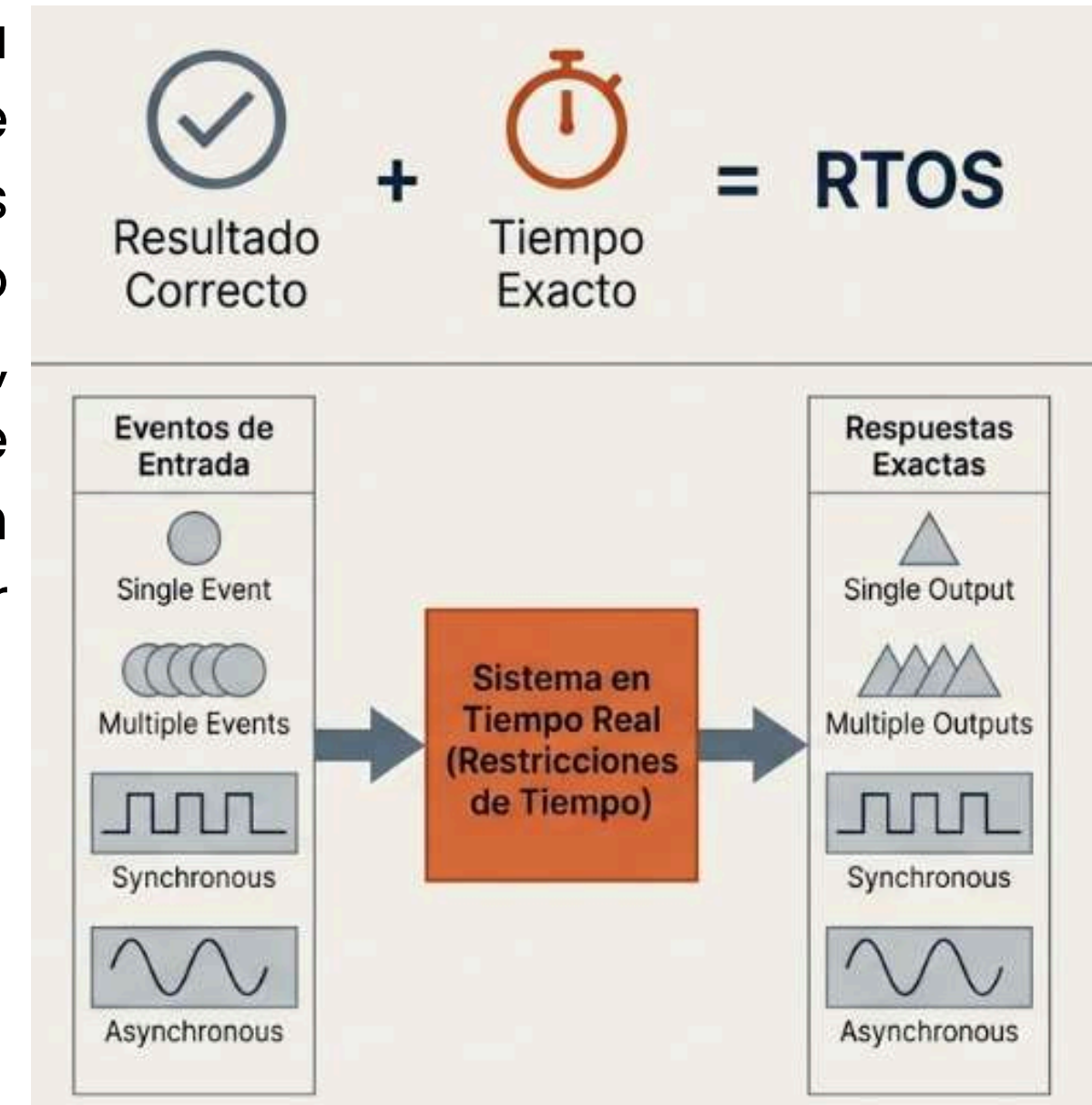
Asignatura: Sistemas Digitales

**Integrantes: Jasiel Quezada, Isauro Rivera,
Benito Minga, Andres Tapia, Jean Encalada**



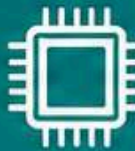
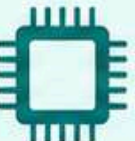
Introduccion a los RTOS

Un RTOS (Real-Time Operating System) es un sistema operativo diseñado para ejecutar tareas dentro de tiempos definidos y predecibles. A diferencia de los sistemas operativos tradicionales, en un RTOS no solo importa que la operación se complete correctamente, sino también que la respuesta ocurra en el instante exacto requerido. Por esta razón, se emplea en aplicaciones críticas donde un retraso puede provocar fallos o afectar el funcionamiento del sistema.



En tiempo real, una respuesta tardía es un error fatal.

DIFERENCIAS

	 SO Tradicional (GPOS)	 Sistema Embebido	 RTOS
 Objetivo	 Maximizar rendimiento general	 Función específica	 Cumplir plazos estrictos (Deadlines)
 Tiempo de Respuesta	 Promedio (No garantizado)	 Rápido (Pero flexible)	 Determinista (Garantizado)
 Uso de Recursos	 Alto (PCs, Servidores)	 Muy bajo (Microcontroladores)	 Optimizado (Kernel pequeño)
 Aplicaciones	 Servidores, PCs	 Electrodomésticos	 Aeroespacial, Automoción

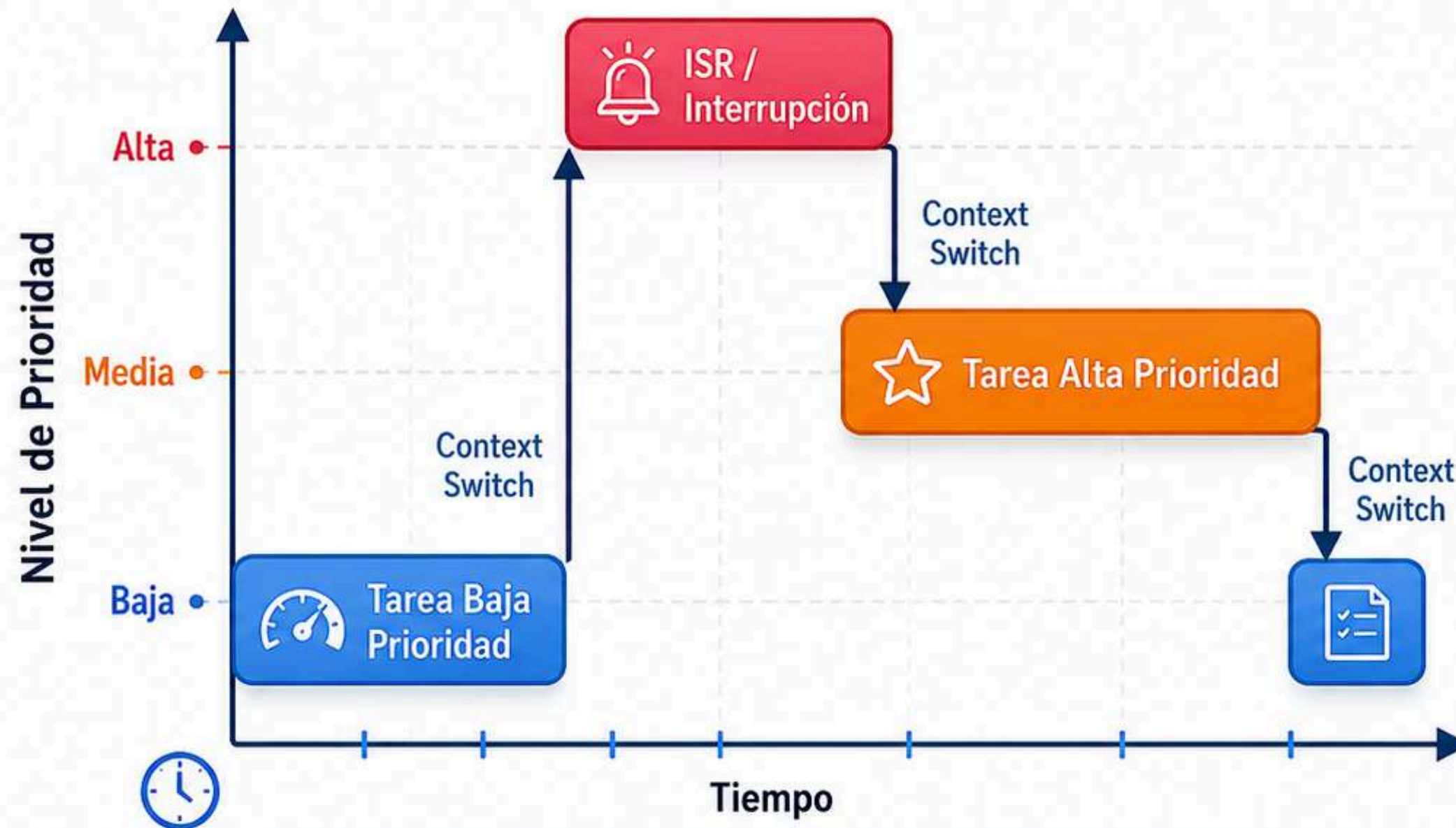
Características principales



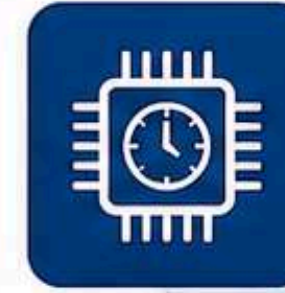
Los RTOS están diseñados para sistemas embebidos donde el **tiempo** y la **confiabilidad** son críticos.

Multitarea y prioridades

Diagrama de Conmutación de Contexto (Planificación Expropiativa)



La planificación expropiativa permite que tareas de **mayor prioridad** interrumpan a las de **menor prioridad**.



Scheduler (Despachador):

Asigna el tiempo de CPU a las tareas según las prioridades.



Planificación Expropiativa:

Tareas de alta prioridad interrumpen a las de baja prioridad.



Prioridades:

Basadas en la criticidad, no en el orden de llegada.



Interrupciones (ISRs):

Manejan eventos asíncronos externos de manera inmediata.



Context Switch:

Cambio de tareas rápido y predecible para garantizar respuesta en tiempo real.

APLICACIONES REALES



1 Automoción

Control de frenos ABS, ADAS y sistemas de seguridad.



2 Aeroespacial

Control de vuelo, estándares DO-178C y sistemas críticos.



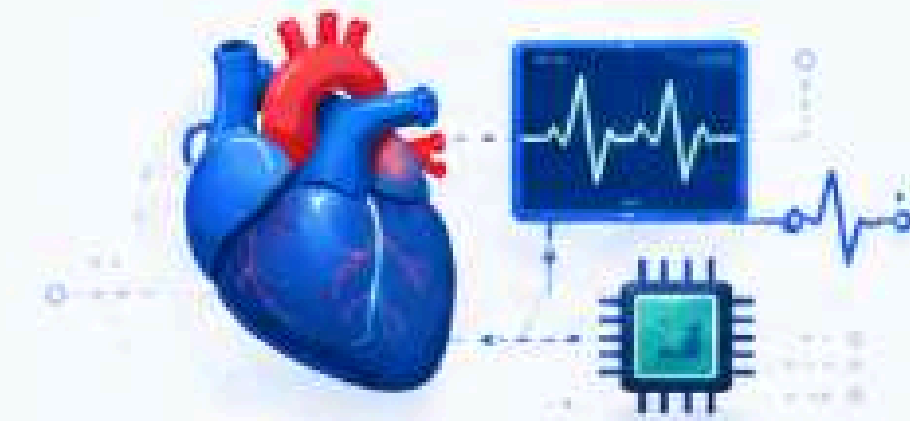
3 Robótica

Cinemática, control de movimiento y brazos industriales.



4 IoT y Edge Computing

Monitoreo de redes, sensores y dispositivos remotos en tiempo real.



5 Medicina

Marcapasos, monitoreo de pacientes y dispositivos médicos críticos.



6 Industria

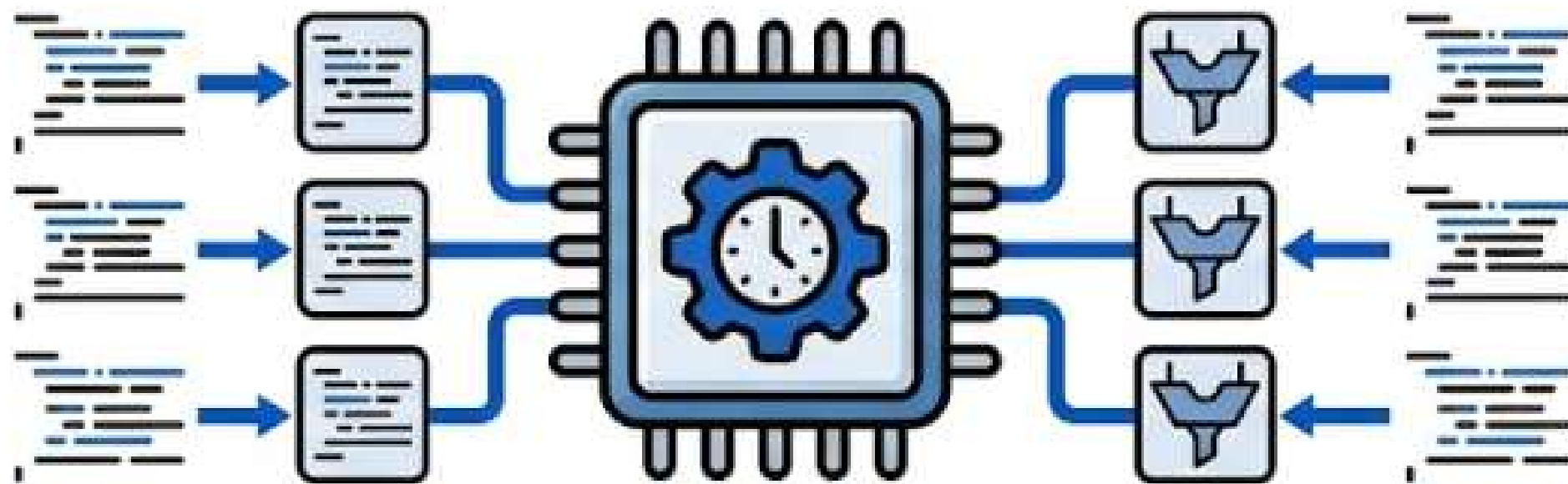
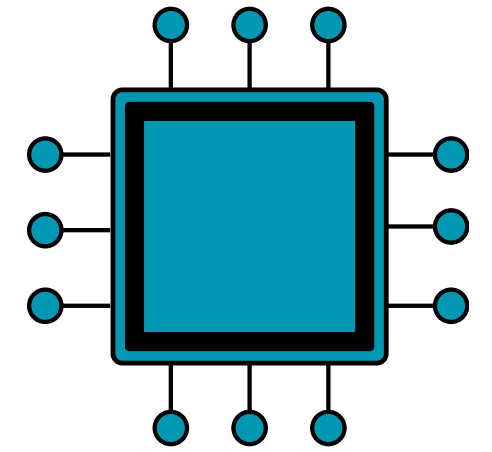
Controladores PLC de doble núcleo, automatización y procesos industriales.

Conceptos Fundamentales

Tareas

Una tarea es un hilo de ejecución independiente que contiene una secuencia de instrucciones y compite con otros por el tiempo de procesamiento de la CPU

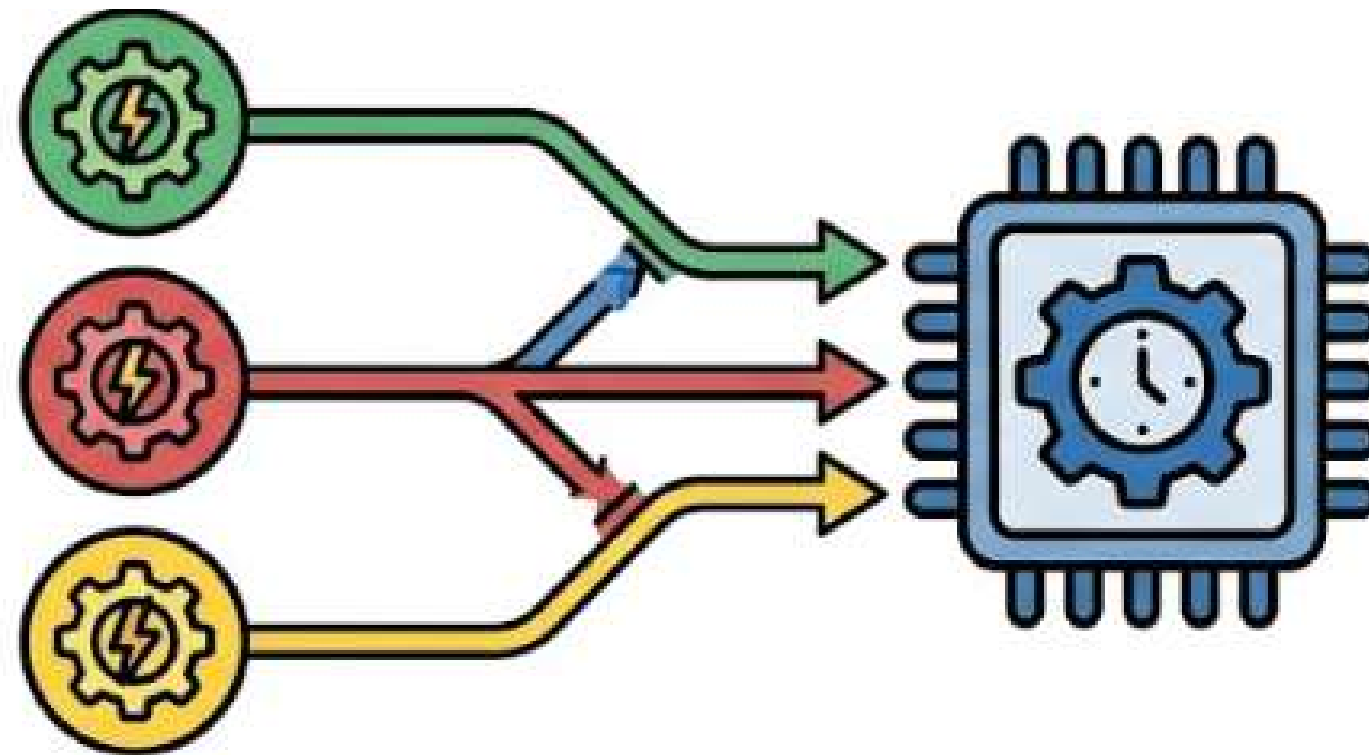
Cada tarea se define como un objeto compuesto por un Bloque de Control de Tarea (TCB), un área de pila (stack) propia, una prioridad y una rutina de código que generalmente es un bucle infinito



Procesos

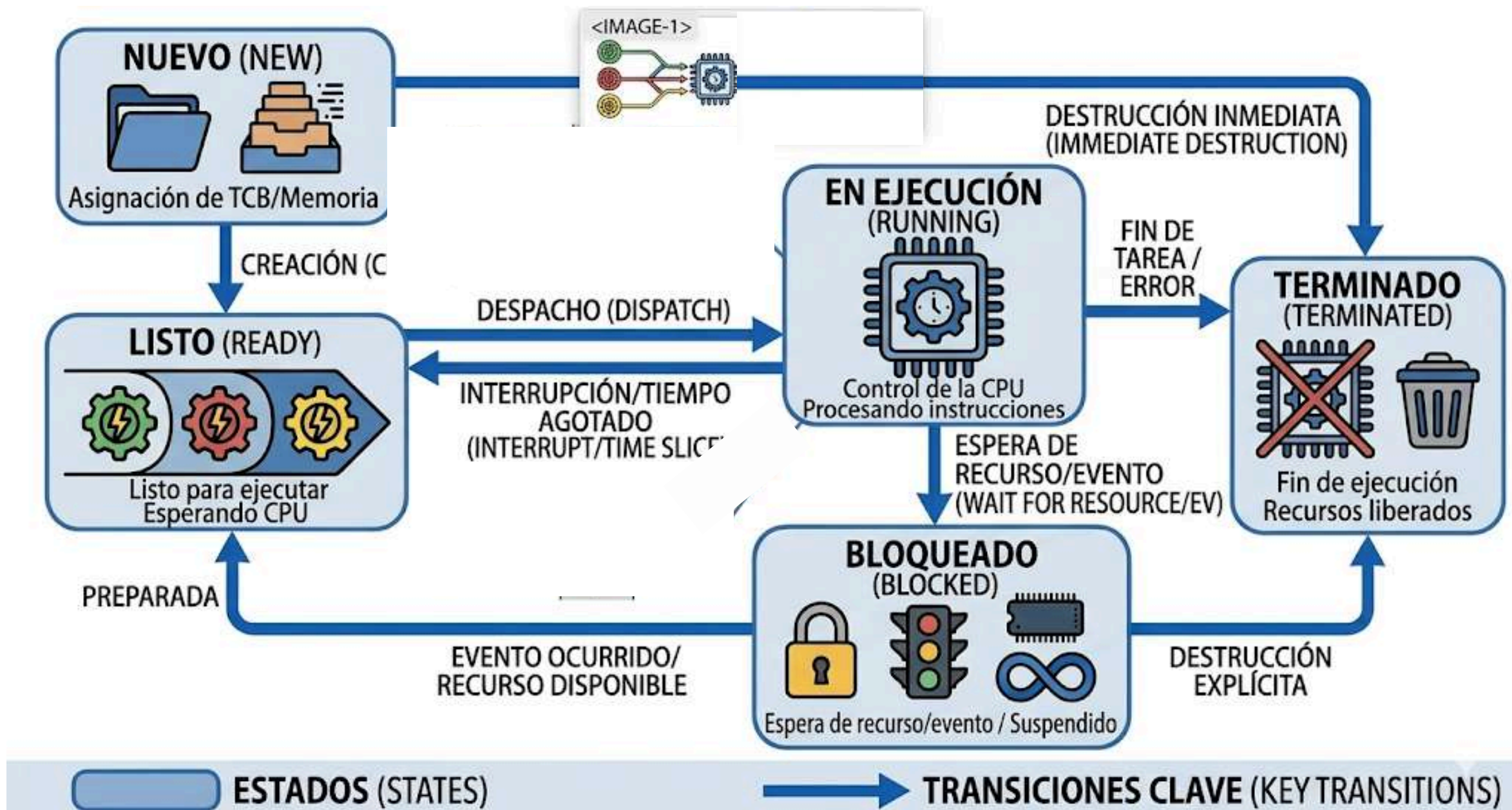
En muchos sistemas operativos de tiempo real, el término "proceso" se utiliza como para describir una computación secuencial.

Es una actividad computacional más compleja que puede estar compuesta por varios hilos, que suele ejecutarse en espacio de direcciones separados y pRotegidos por el hardware.



Estados de una tarea

El kernel utiliza una máquina de estados para rastrear en qué situación se encuentra cada tarea. Los estados principales son:

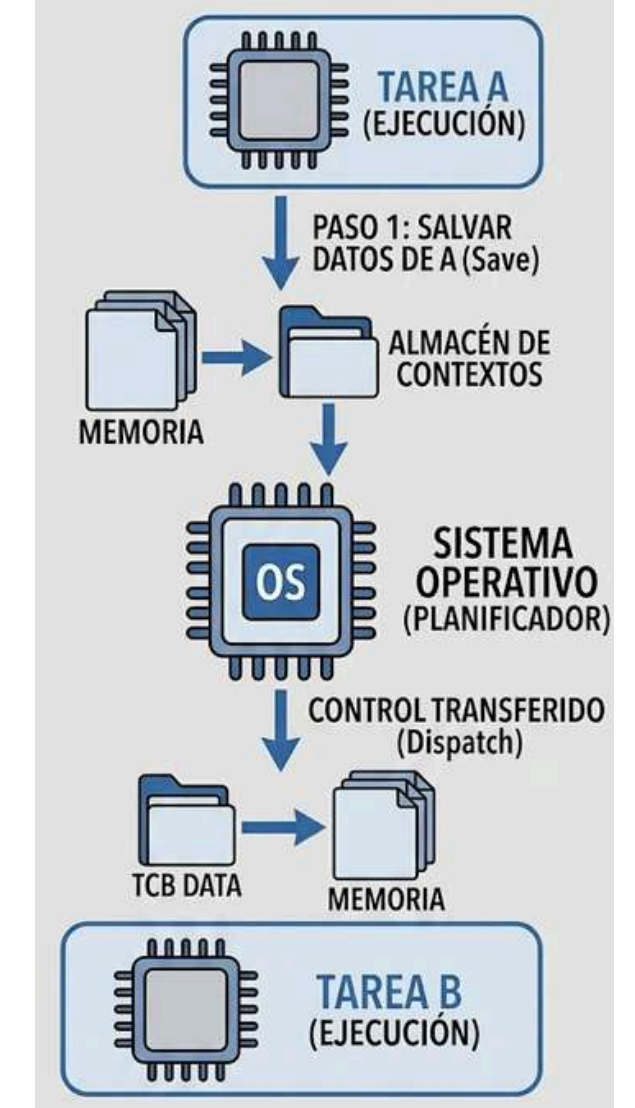


- **Running (En ejecución)**: La tarea tiene actualmente el control de la CPU y sus instrucciones se están procesando.
- **Ready (Listo)**: La tarea está preparada para ejecutarse, pero espera su turno porque otra tarea tiene el control del procesador.
- **Blocked (Bloqueado)**: La tarea se suspende a sí misma para esperar un recurso no disponible (como un semáforo), un evento externo o la expiración de un tiempo de retardo.
- **New(Nuevo)**: La tarea ha sido definida y su bloque de control de tarea(TCB) está asignado en la memoria.
- **Terminated(Terminado)**: La tarea finalizó con su ejecución o fue destruida explícitamente.

Cambio de contexto (Context Switching)

El contexto es el conjunto de datos que describe el estado exacto del procesador en un instante determinado durante la ejecución de una tarea.

Consiste en congelar la ejecución de una tarea actual para transferir el control de la CPU a otra tarea.



Prioridades

Número asignado a cada tarea que indica su importancia o urgencia con la que debe ser asistida.

Las prioridades pueden ser estáticas o dinámicas para manejar situaciones como la prioridad escalable.

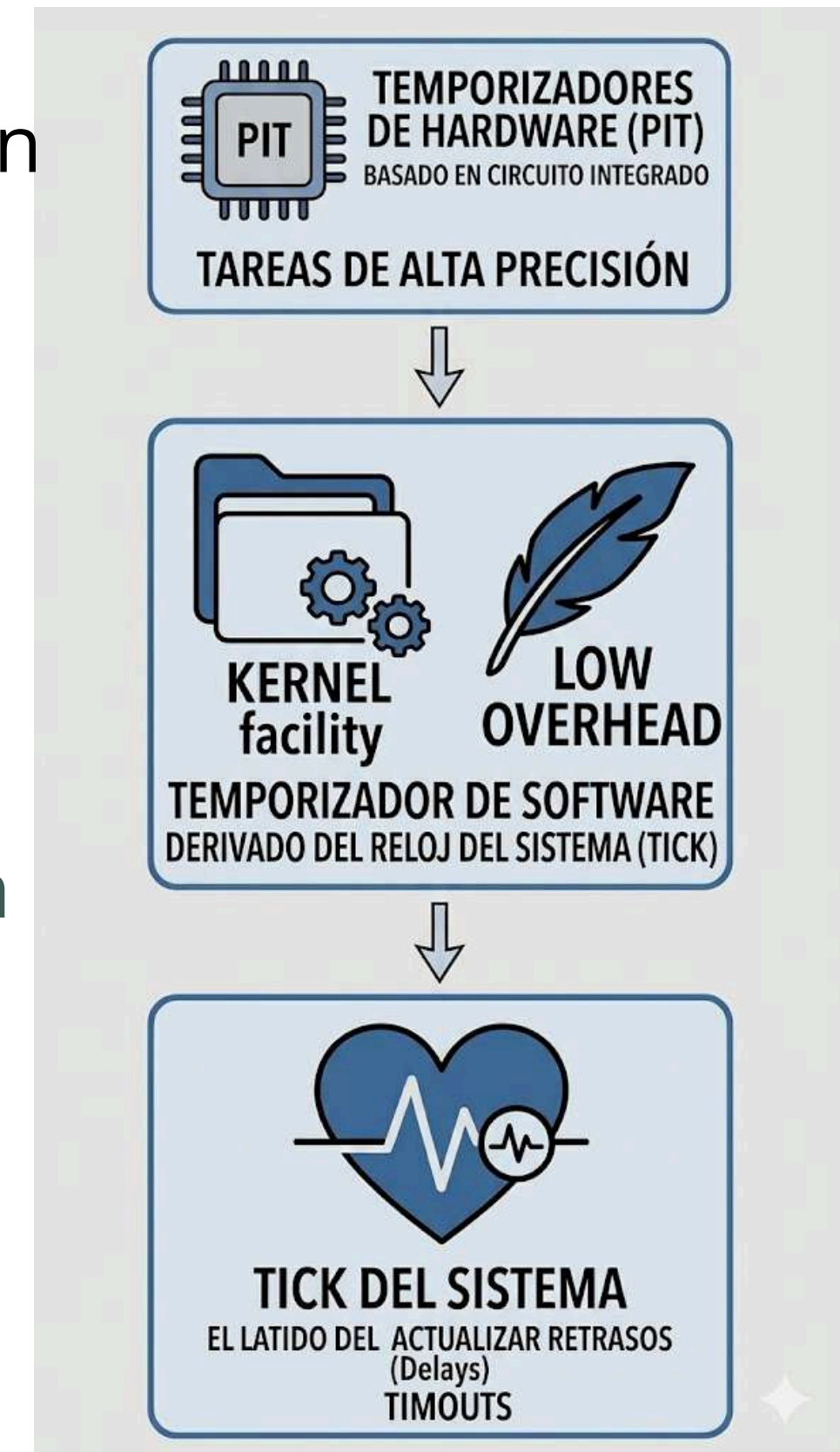


Temporizadores



Los temporizadores permiten programar actividades para que ocurran en un momento específico del futuro. Se clasifican en:

- **Temporizadores de Hardware (Hard Timers):** Basados en circuitos integrados (PIT) que interrumpen directamente al procesador para tareas de alta precisión.
- **Temporizadores de Software (Soft Timers):** Eventos programados a través de una facilidad del kernel que se derivan del reloj del sistema (tick); son menos precisos pero reducen la sobrecarga de interrupciones de hardware.
- **Tick del sistema:** Es el "latido" del kernel, una interrupción periódica que sirve para actualizar retrasos, tiempos de espera (timeouts) y realizar planificación por rotación (round-robin).



Interrupciones (ISR)

Rutinas de Servicio de Interrupción permiten que un RTOS responda inmediatamente a eventos del hardware, interrumpiendo temporalmente la ejecución normal de las tareas.

Funcionamiento general

Cuando ocurre una interrupción, el procesador:

- Guarda el estado actual de la CPU.
- Busca la dirección de la ISR mediante la tabla de vectores.
- Ejecuta la rutina correspondiente.
- Retorna a la tarea anterior o cambia a otra de mayor prioridad.

Jerarquía de prioridad

Las interrupciones poseen mayor prioridad que las tareas del sistema.

Una ISR puede suspender incluso la tarea más importante para atender eventos críticos del hardware.

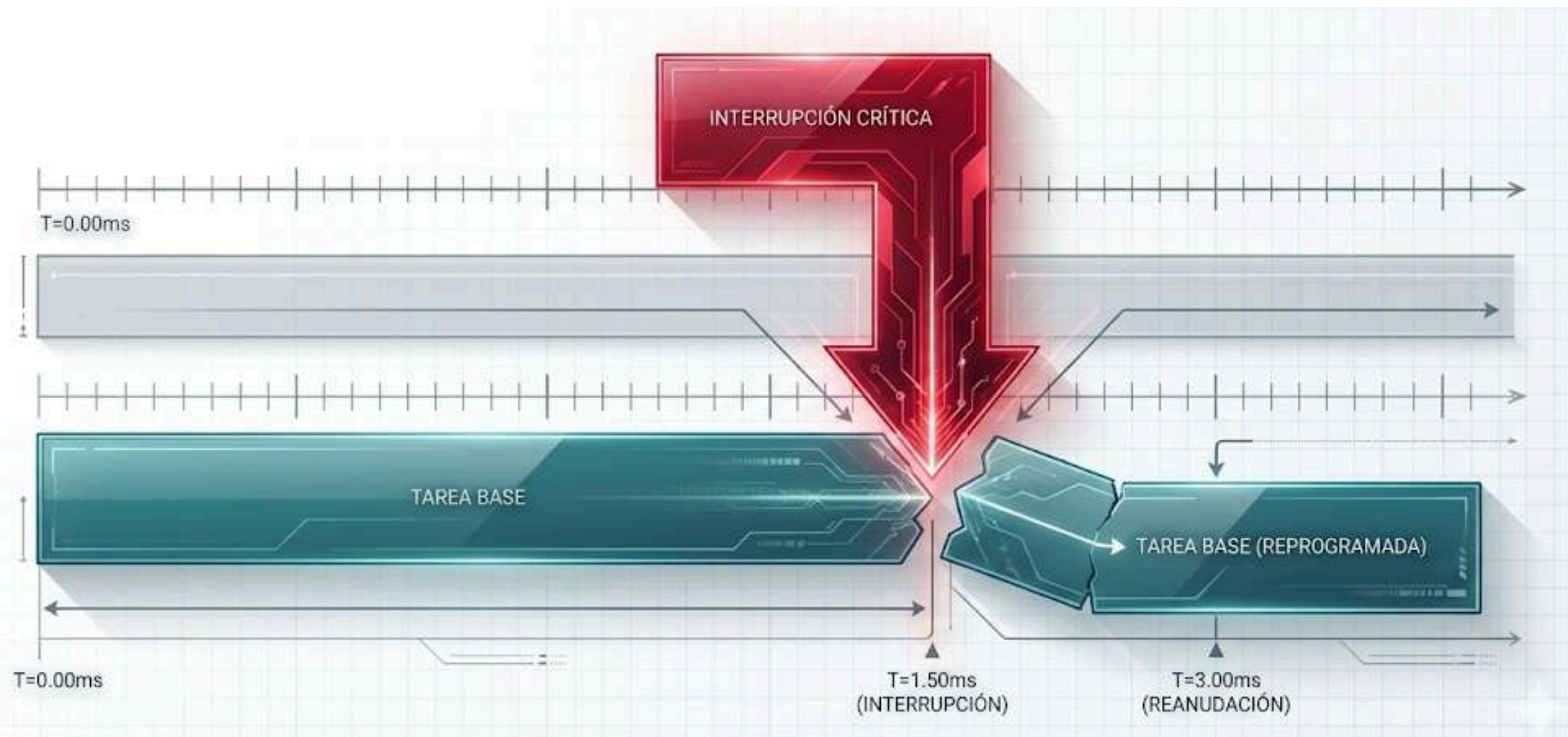


Digital Transformation

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam sit amet euismod tellus. Vestibulum interdum a sem id faucibus. Nunc velit eros, euismod id tempus vel, facilisis ac sapien. Curabitur tincidunt consequat ex id aliquet. Nunc in erat et arcu vestibulum dapibus sed in mauris. Donec egestas ornare leo, commodo scelerisque nisl interdum vel. Fusce non dui quis lectus bibendum semper. Morbi lacinia mauris eget tristique vehicula. Proin sollicitudin, neque vel lobortis egestas, velit turpis rhoncus sapien, vulputate ultrices metus nisi nec urna. Maecenas id maximus libero, ac luctus ipsum. Pellentesque id turpis erat. Nulla eu tortor congue, pretium dolor in, finibus nibh.



Planificación en tiempo real



Es el mecanismo principal de un RTOS, responsable de decidir qué tarea se ejecuta y cuándo, para garantizar que se cumplan las restricciones temporales del sistema.

Scheduling en RTOS

El planificador (scheduler) o despachador (dispatcher) es el componente del núcleo que asigna la CPU a las diversas tareas siguiendo una política o algoritmo de planificación; la diferencia es que para scheduler su objetivo principal no es el rendimiento promedio, sino la **previsibilidad** y el cumplimiento de los plazos (deadlines)

Scheduler (Planificador)

Es el encargado de:

- Organizar tareas.
- Asignar tiempo de CPU.
- Decidir prioridades.

Su objetivo es garantizar
que las tareas críticas
cumplan sus tiempos límite
(deadlines)



Tipos de planificación

Planificación Preventiva (Preemptive)

Es el mecanismo donde el núcleo puede interrumpir una tarea en ejecución para entregar la CPU a otra tarea de mayor prioridad que acaba de estar lista.



Planificación Cooperativa (Non-Preemptive)

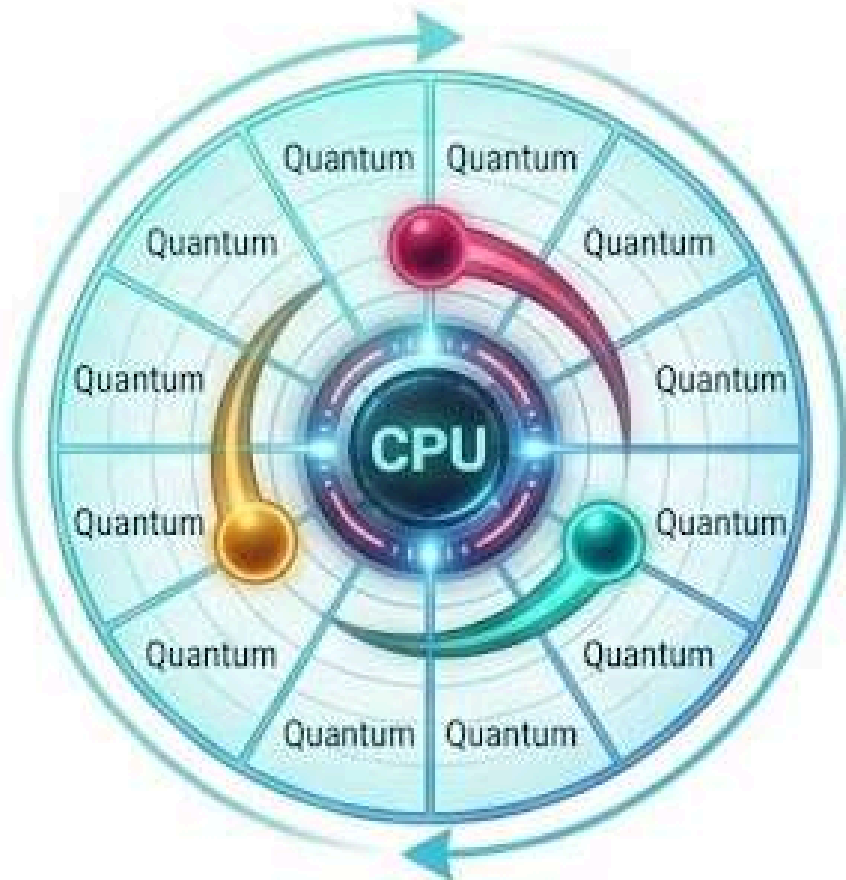
Aquí, una tarea mantiene el control de la CPU hasta que decide voluntariamente cederlo o termina su ejecución. Su respuesta ante tareas urgentes es lenta y no determinista.



Algoritmos comunes

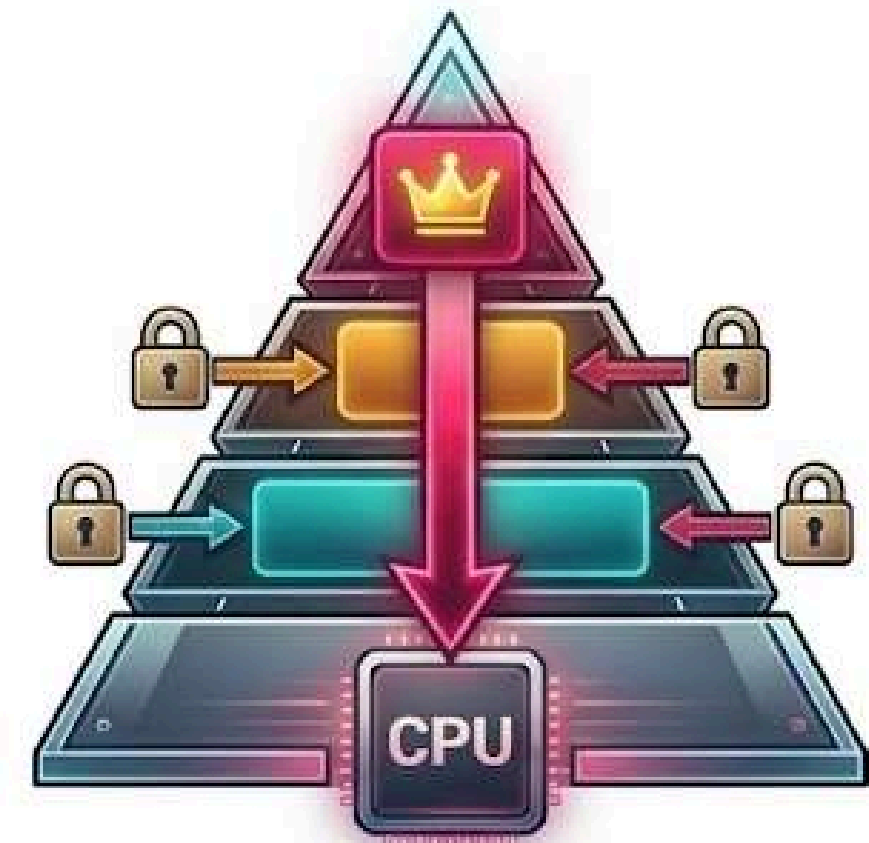
Round Robin (RR)

Se utiliza principalmente para tareas con la misma prioridad. El sistema asigna un intervalo de tiempo fijo (quántum) a cada tarea para que utilicen la CPU por turnos.



Priority Scheduling

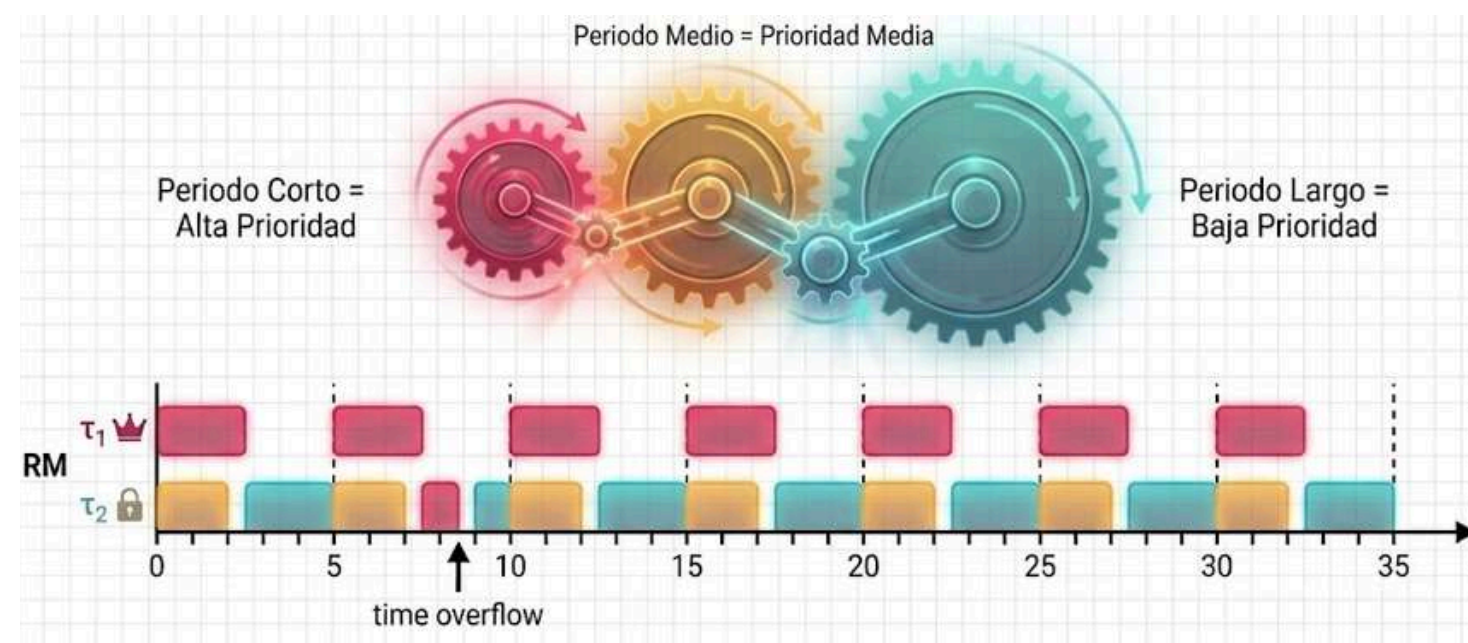
A cada tarea se le asigna una prioridad basada en su importancia. El planificador siempre ejecuta la tarea con la prioridad más alta que esté en estado "listo" (ready).



Algoritmos comunes

Rate Monotonic Scheduling (RMS)

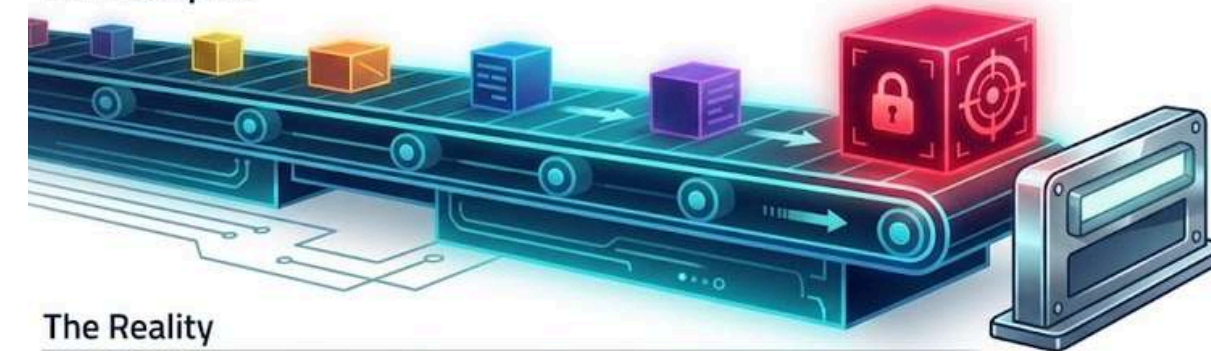
Es un algoritmo de prioridad fija donde las tareas con los periodos más cortos (mayor frecuencia) reciben las prioridades más altas. Se basa en la premisa de que las tareas que ocurren más seguido son más críticas



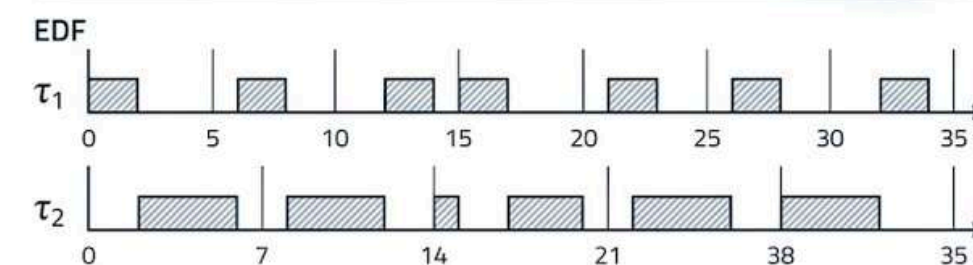
Earliest Deadline First (EDF)

Es un algoritmo de prioridad dinámica que asigna la mayor prioridad a la tarea cuya fecha límite (deadline) absoluta sea la más cercana en el tiempo.

The Metaphor



The Reality

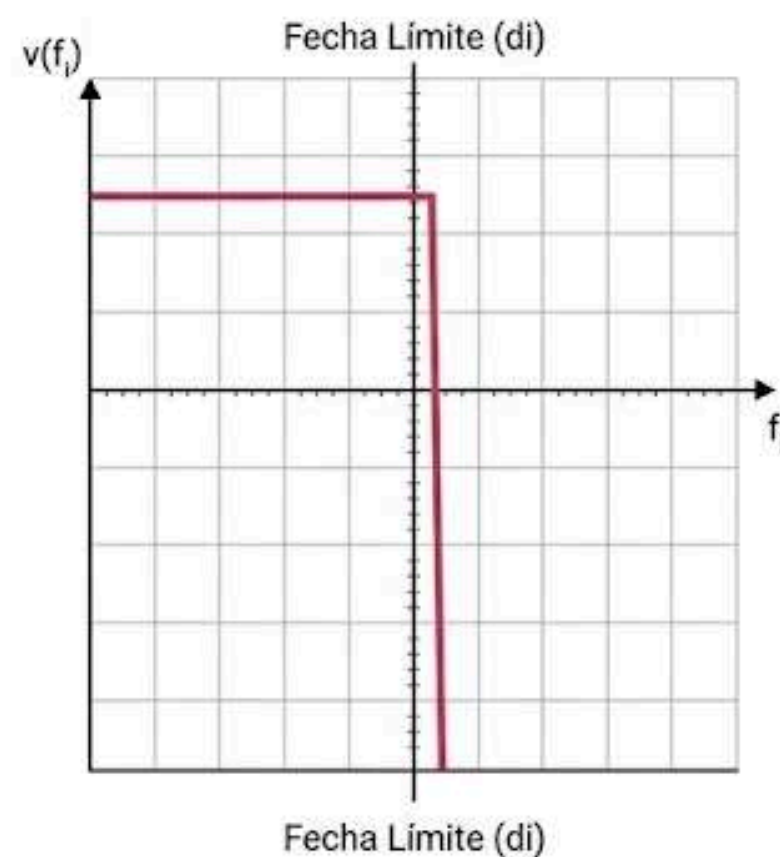


Mecanismo Dinámico:
La fecha límite más cercana dicta el control total. Permite hasta el 100% de utilización de la CPU.

Categorías de tiempo real

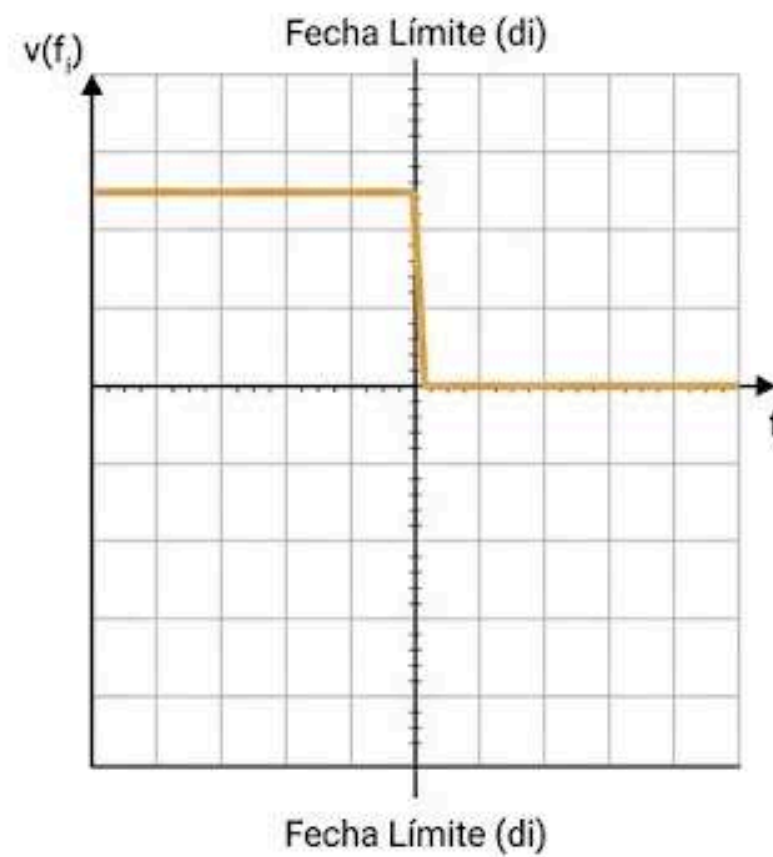
Hard Real-Time

El incumplimiento de un solo plazo puede causar consecuencias catastróficas, como daños físicos, pérdida de vidas o del sistema.



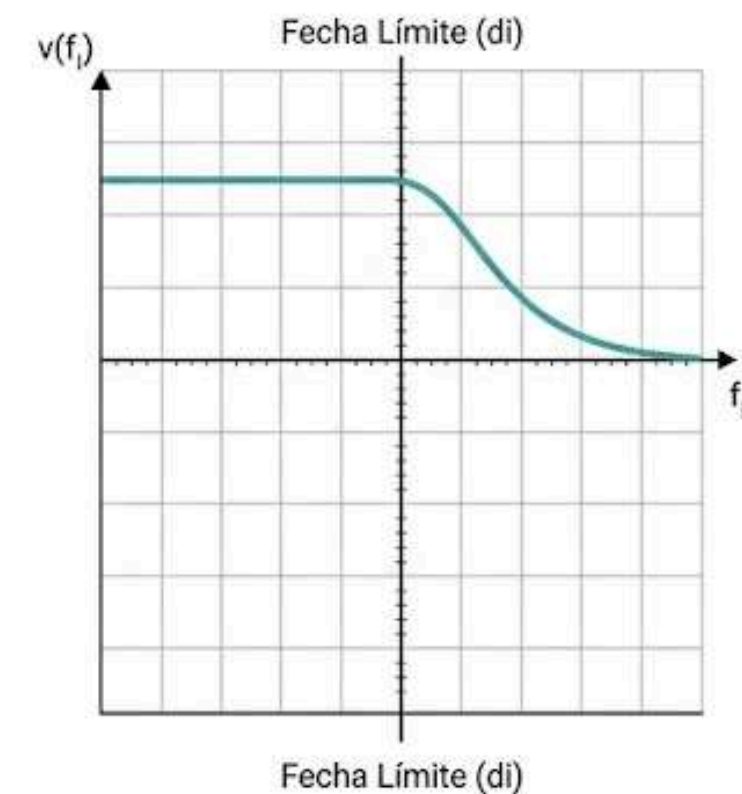
Firm Real-Time

Si se pierde un plazo, el resultado es inútil para el sistema, pero no causa daños inmediatos.



Soft Real-Time

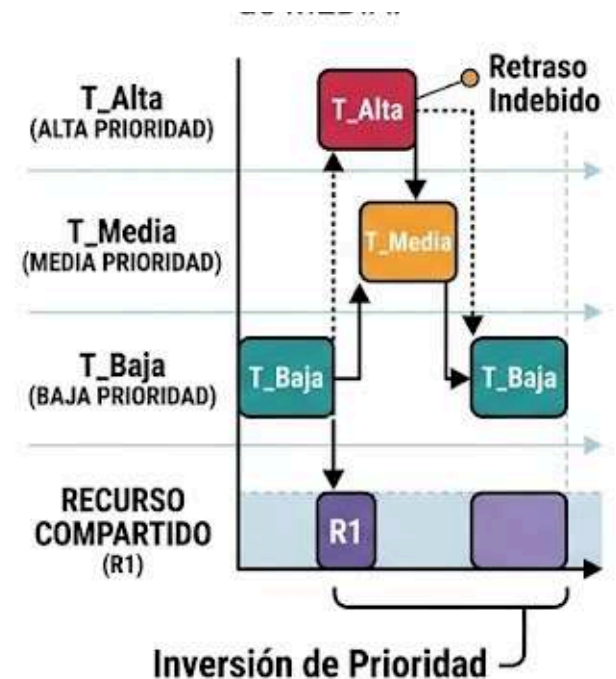
Los plazos son importantes pero no obligatorios. Si se incumple uno, el sistema sigue funcionando pero con una degradación en la calidad del servicio o rendimiento.



Problemas comunes

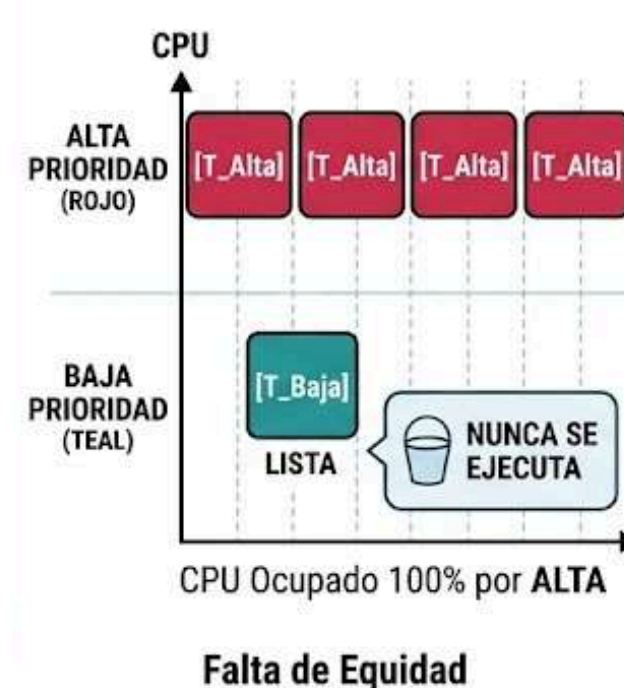
Inversión de prioridad

Ocurre cuando una tarea de alta prioridad es bloqueada por una de baja prioridad que posee un recurso compartido, y una tarea de prioridad media interrumpe a la de baja, retrasando indefinidamente a la de alta.



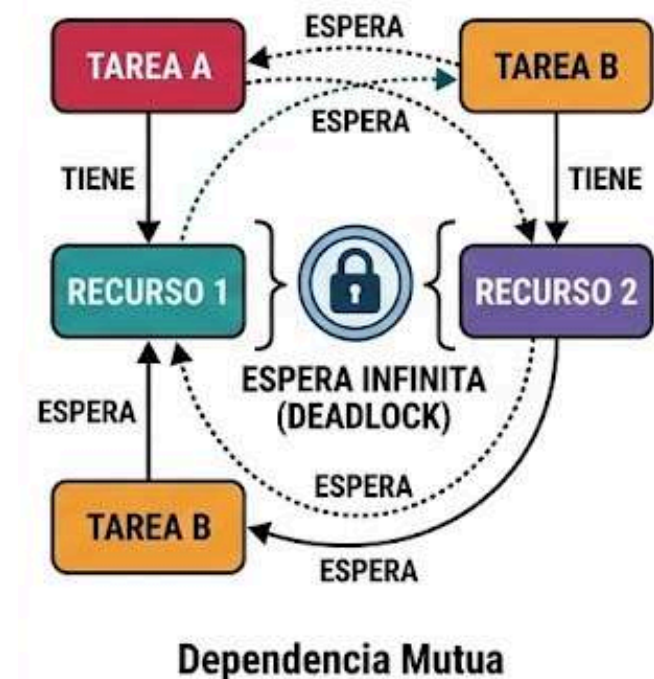
Starvation (Inanición)

Sucede cuando las tareas de alta prioridad consumen todo el tiempo de CPU, impidiendo que las tareas de menor prioridad se ejecuten nunca.



Deadlocks (Interbloqueos)

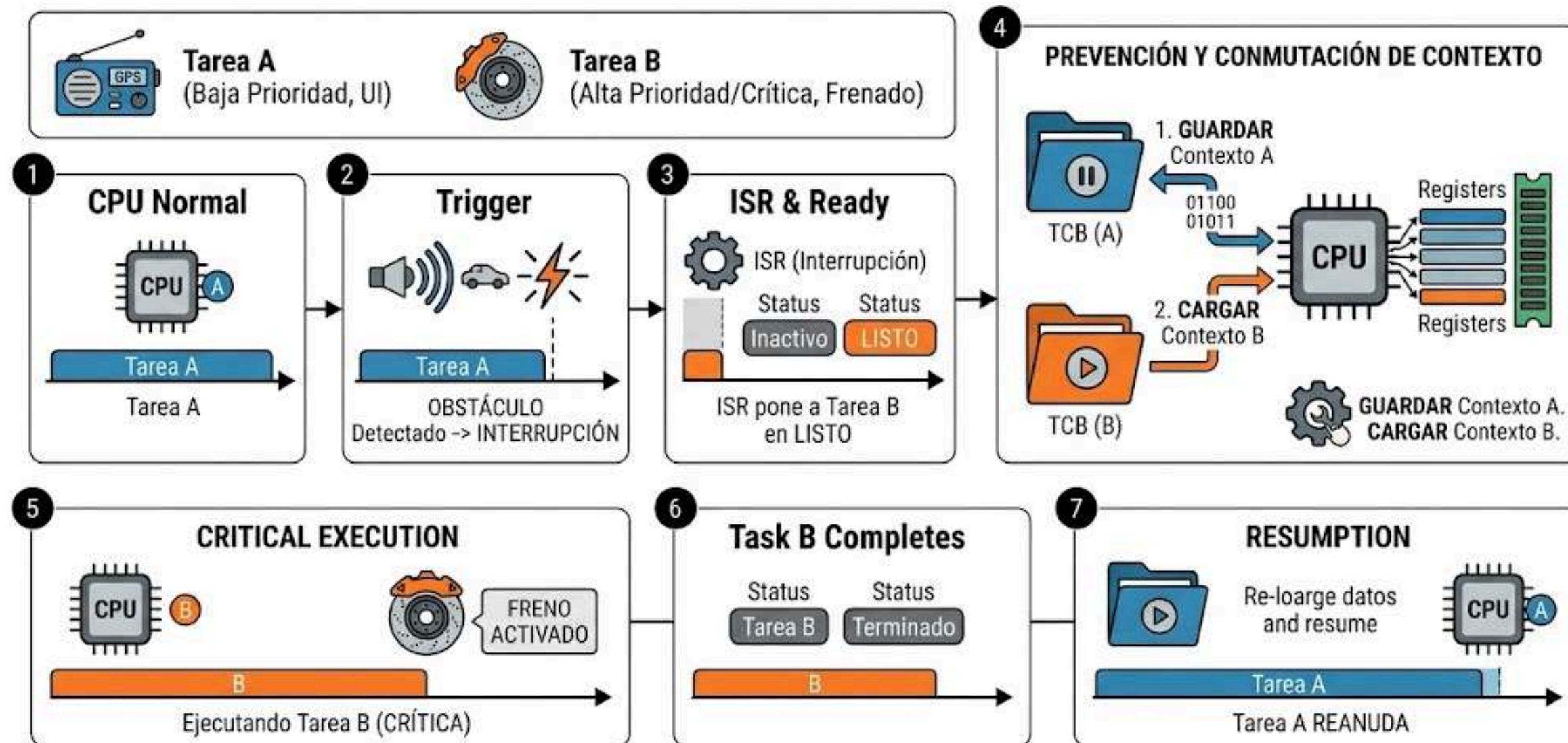
Situación donde dos o más tareas se bloquean mutuamente esperando recursos que la otra posee, creando una espera infinita.



Ejemplo: Interrupción de tarea crítica

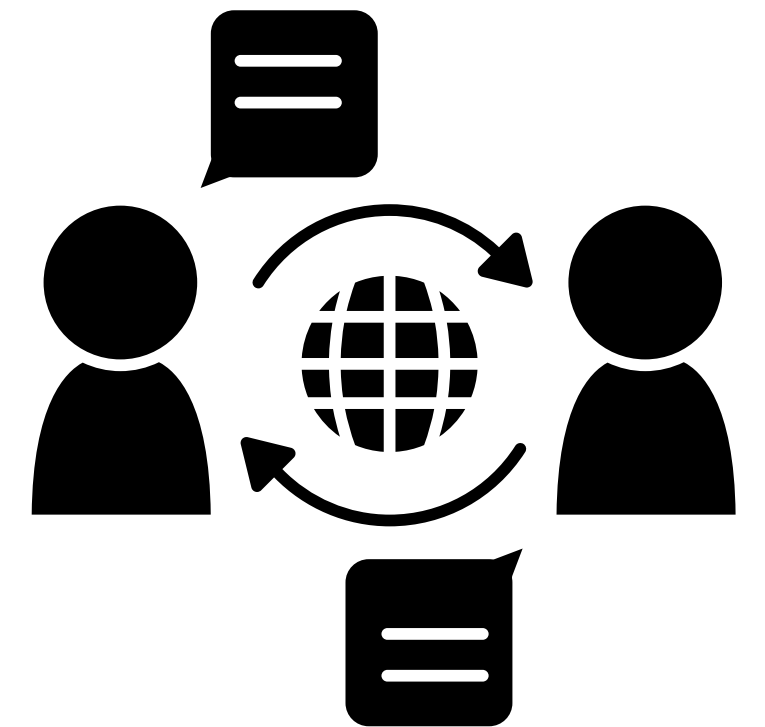
Imagina un sistema con una Tarea A (Baja prioridad) controlando la interfaz de usuario y una Tarea B (Alta prioridad/Crítica) gestionando un sensor de frenado.

- La Tarea A está ejecutándose normalmente.
- Ocurre un evento externo (sensor de proximidad activado) que genera una interrupción (ISR).
- La ISR maneja el evento y pone a la Tarea B en estado "listo".
- Como el sistema es preventivo, el núcleo guarda el contexto de la Tarea A, realiza una conmutación de contexto inmediata y comienza a ejecutar la Tarea B para responder al frenado de emergencia.
- Solo cuando la Tarea B termina o se bloquea, la Tarea A puede reanudar su ejecución





Comunicación y Sincronización



¿Qué es?

La comunicación y sincronización permiten que las tareas de un sistema operativo en tiempo real trabajen de manera coordinada y segura. Gracias a estos mecanismos, las tareas pueden compartir información, ejecutar procesos en el momento correcto y evitar conflictos al acceder a recursos del sistema.



Comunicación entre tareas (IPC)

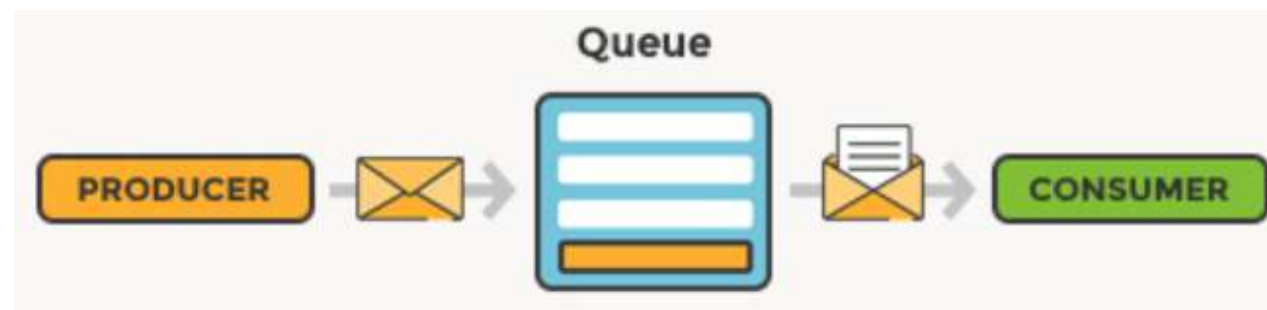
La comunicación entre tareas, conocida como IPC o Inter Process Communication, permite que diferentes procesos compartan información sin necesidad de ejecutarse exactamente al mismo tiempo. Esto mejora la organización y la eficiencia del sistema.



Métodos principales:

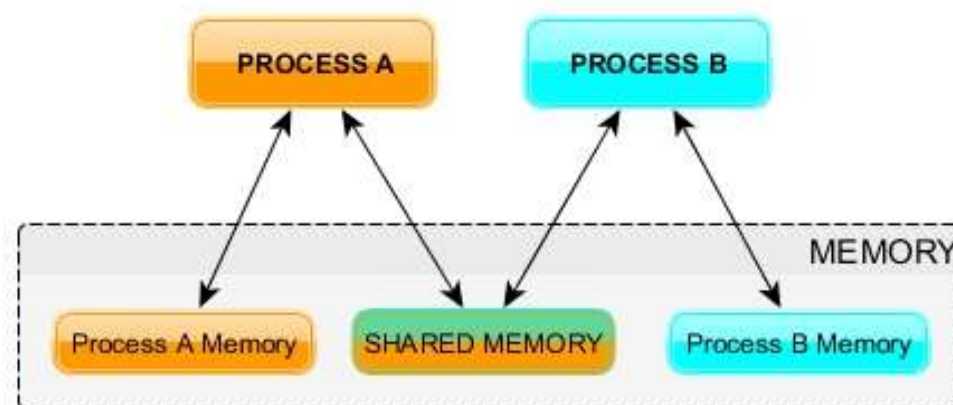
Colas de mensajes

Permiten enviar datos de una tarea a otra siguiendo un orden FIFO, es decir, el primero en entrar es el primero en salir.



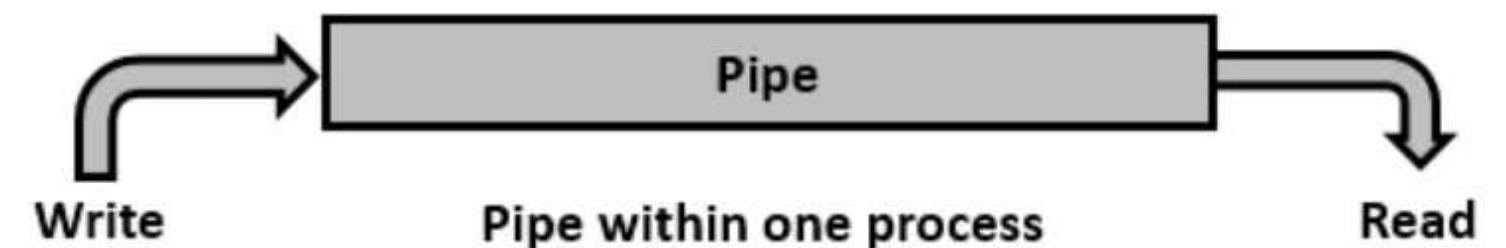
Memoria compartida

Varias tareas acceden a una misma zona de memoria, logrando una comunicación más rápida, aunque requiere control para evitar errores.



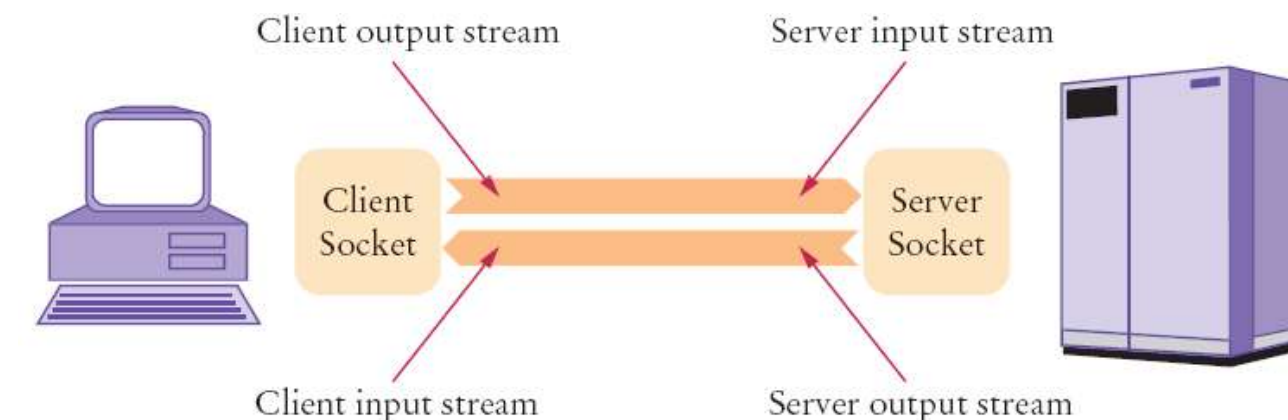
Pipes o tuberías

Son canales de comunicación secuencial y unidireccional que transmiten flujos continuos de datos entre tareas.



Sockets

Permiten la comunicación bidireccional entre procesos o dispositivos a través de una red, facilitando el intercambio de datos entre diferentes sistemas.



Sincronización de tareas

La sincronización controla el orden de ejecución de las tareas y evita que varias accedan al mismo recurso al mismo tiempo. Esto es importante para mantener la estabilidad y evitar pérdida de información.

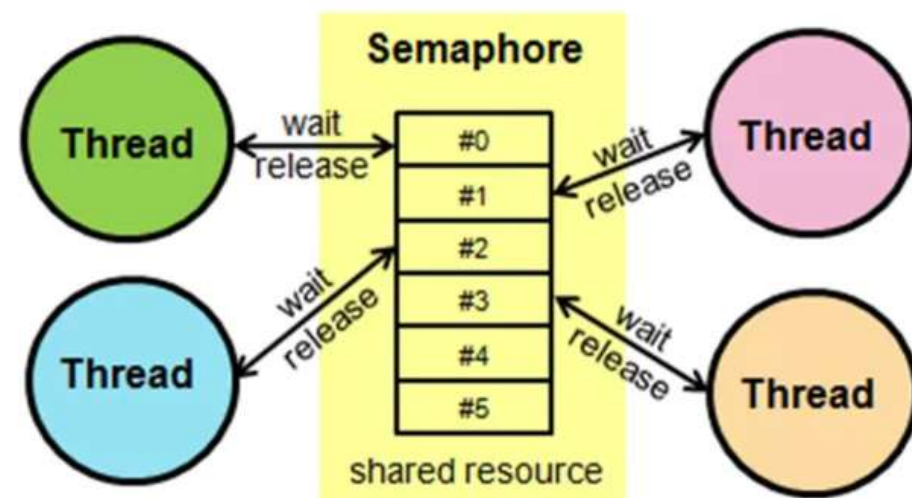
Objetivos:

- Evitar conflictos
- Controlar recursos
- Mantener estabilidad
- Garantizar precisión

Métodos de sincronización:

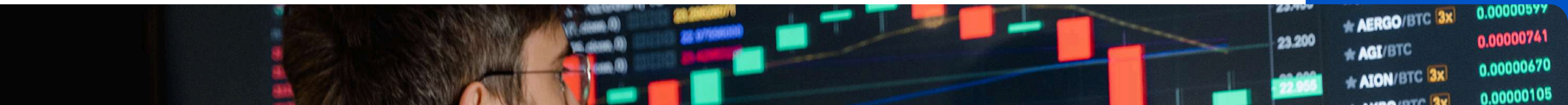
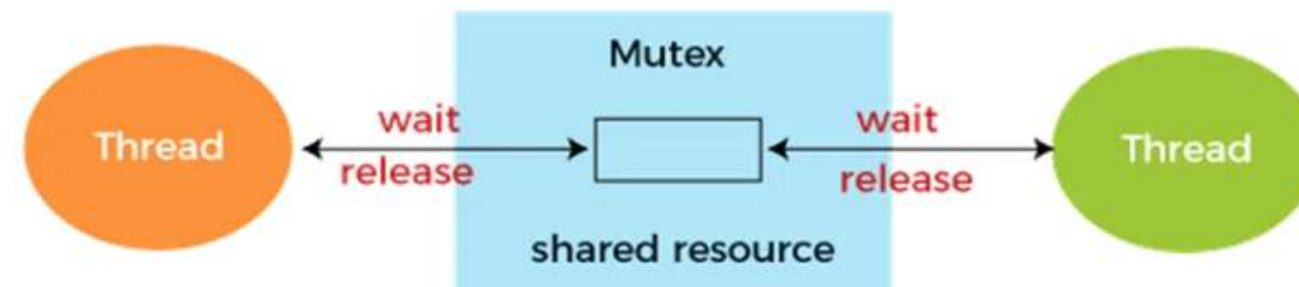
Semáforos

Controlan el acceso a recursos compartidos mediante contadores o señales, permitiendo coordinar la ejecución de múltiples tareas.



Mutex

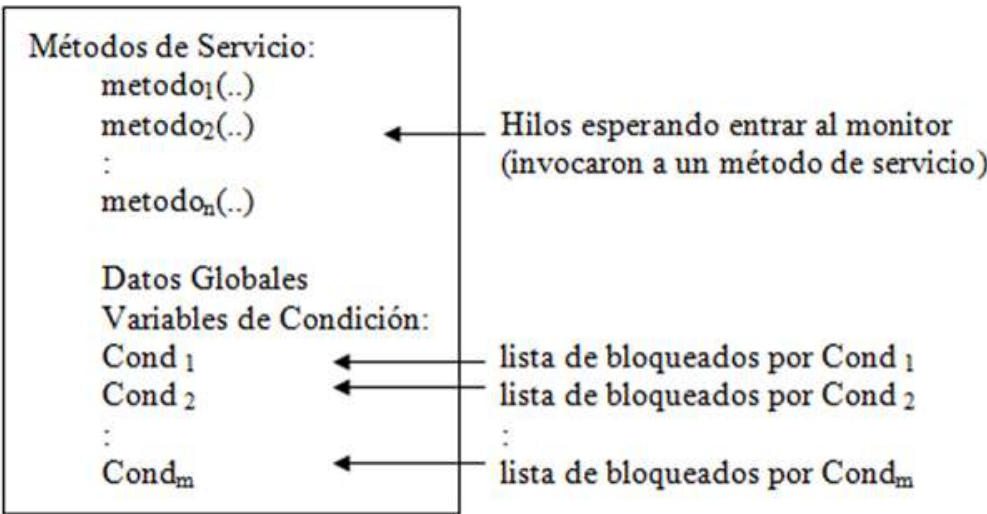
Funcionan como un mecanismo de exclusión mutua que permite que solo una tarea acceda a un recurso crítico al mismo tiempo.



Métodos de sincronización

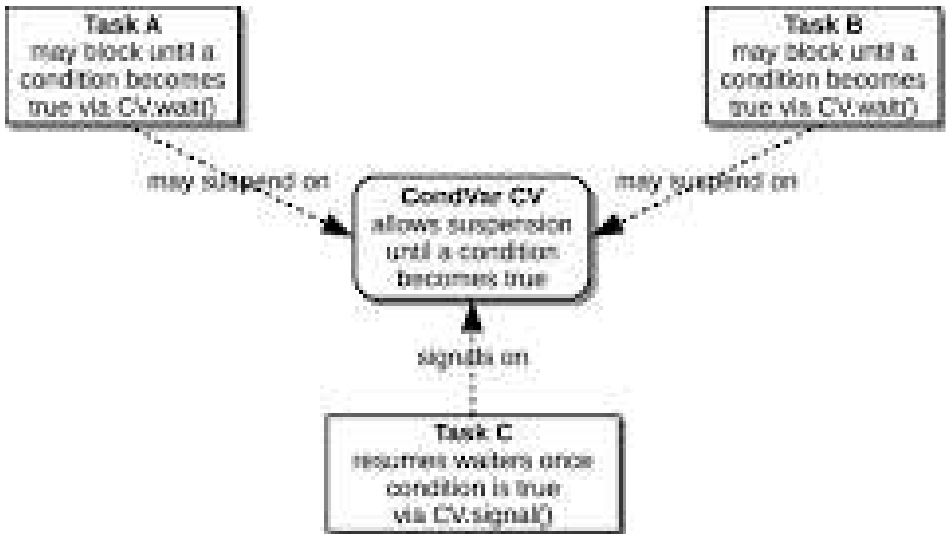
Monitores

Encapsulan datos y funciones compartidas en una sola estructura, garantizando que únicamente una tarea pueda ejecutarse dentro del monitor.



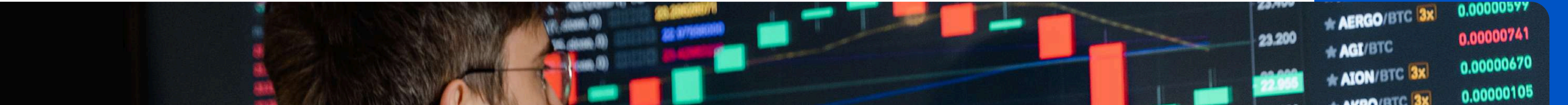
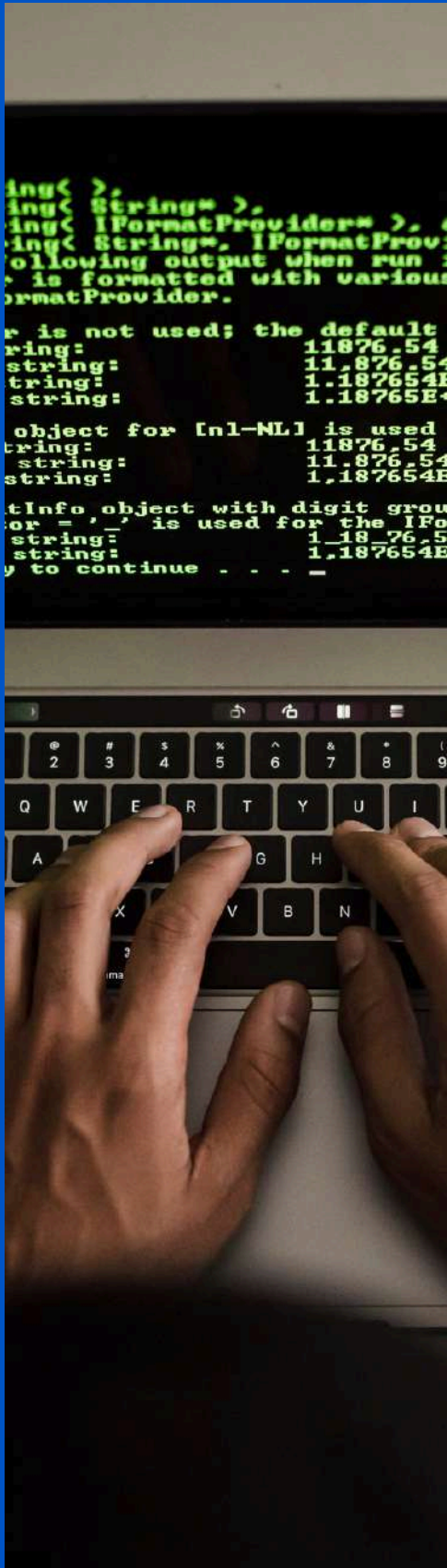
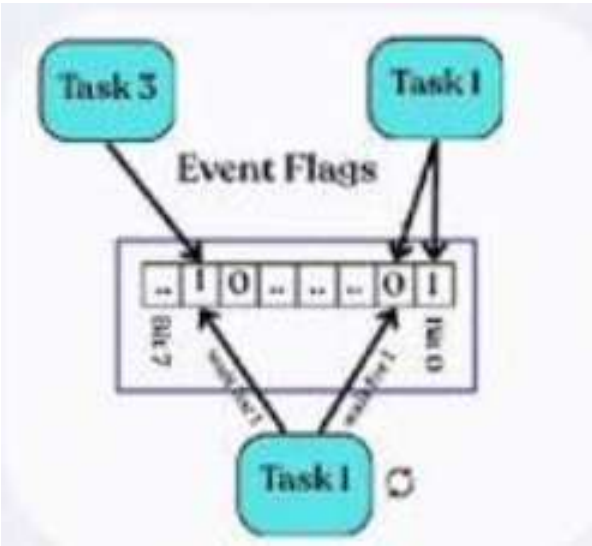
Variables de condición

Permiten que una tarea permanezca en espera hasta que se cumpla una condición específica dentro del sistema.



Eventos

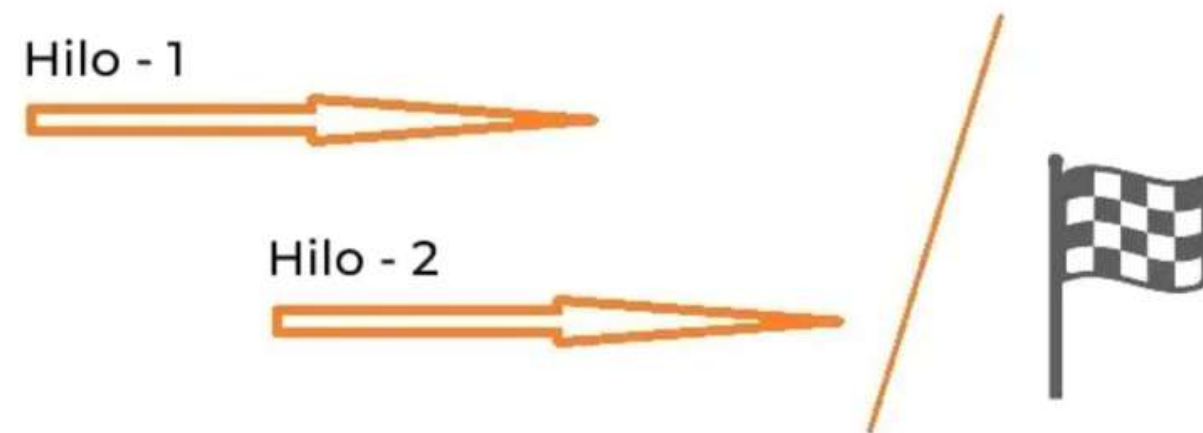
Utilizan señales o banderas lógicas para activar tareas cuando ocurre una condición o evento determinado.



Problemas en sincronización

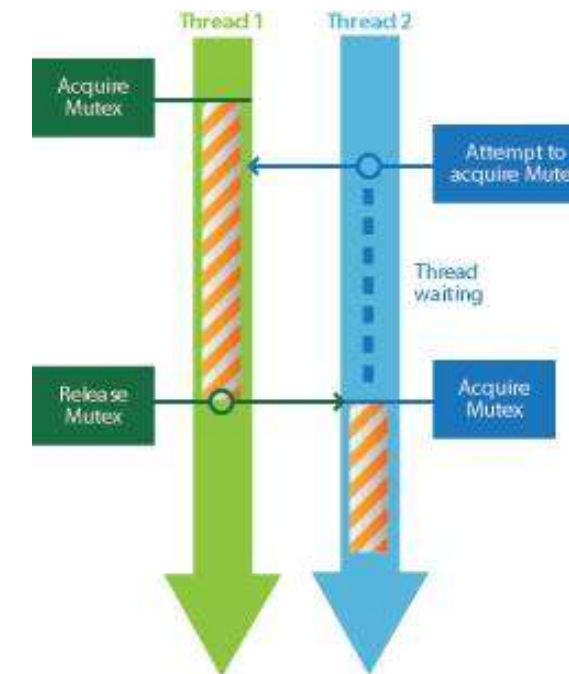
Condición de carrera

Ocurre cuando varias tareas acceden al mismo recurso al mismo tiempo, causando errores o resultados inesperados.



Exclusión mutua

Permite que solo una tarea acceda a un recurso crítico simultáneamente para evitar conflictos.



Ejemplo Productor-Consumidor

Productor

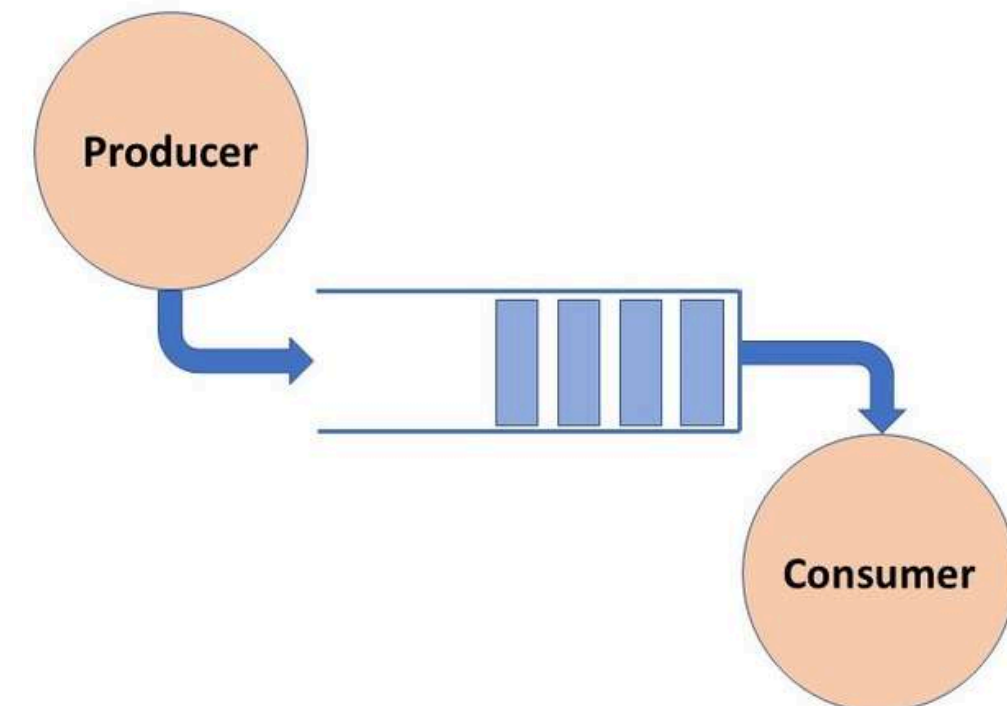
Genera datos y los coloca en una cola de mensajes.

Consumidor

Recibe los datos y los procesa cuando están disponibles.

Sincronización

Los semáforos o mutex evitan conflictos al acceder a la cola compartida.



Gestión de Memoria en RTOS

Memoria Estatica vs Dinamica

Estatica

Se asigna en tiempo de compilacion o antes de que inicie el planificador

Dinamica

Se asigna durante la ejecucion de un programa

Ofrece flexibilidad en entornos donde los requisitos de memoria varían según el estado del sistema

Heap y Stack

Stack

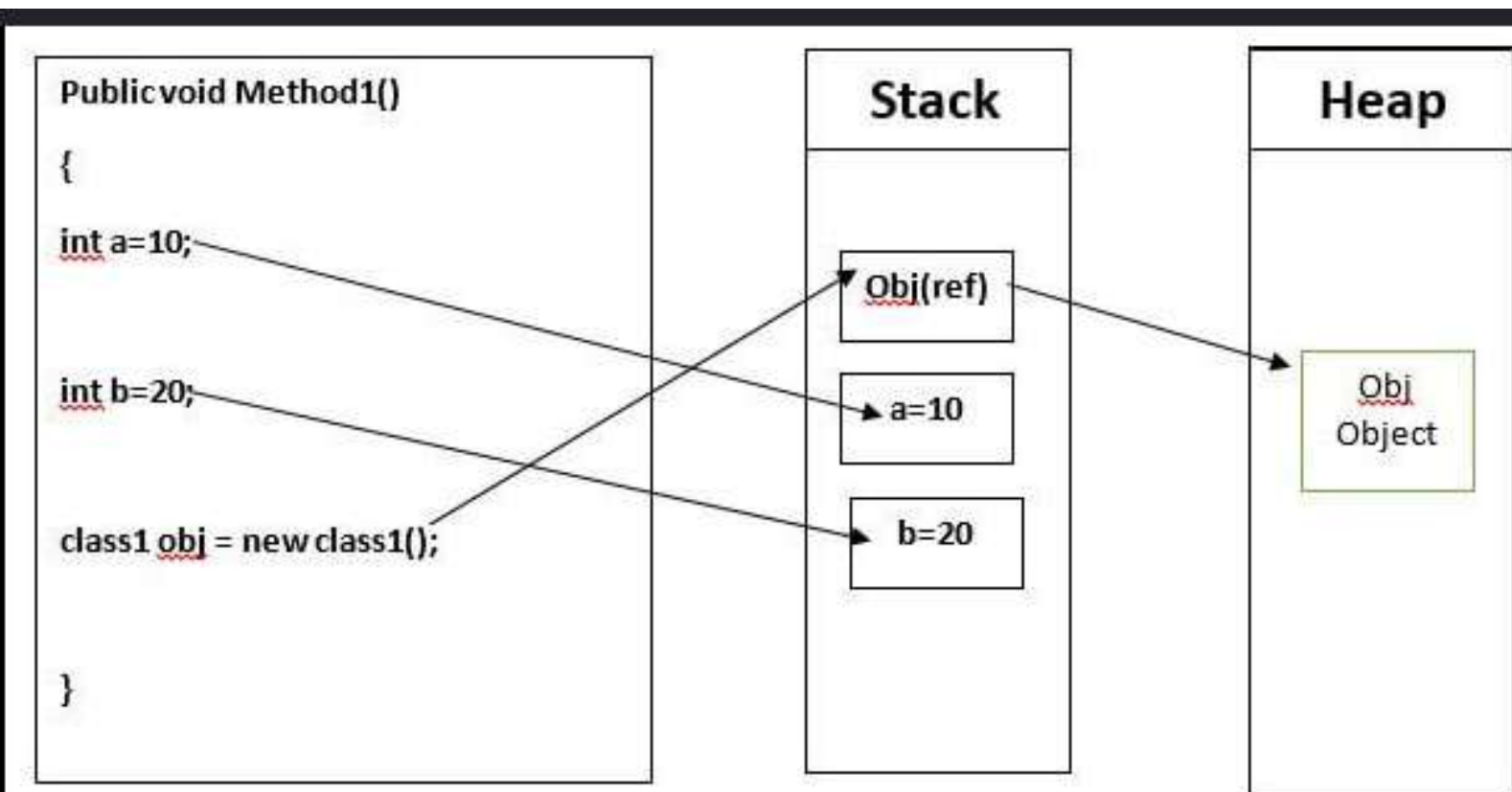
Area de memoria gestionada automáticamente para almacenar variables locales

Funciona bajo el principio LIFO

Heap

Area de memoria restante utilizada por el kernel para la asignación dinámica de memoria

Guarda objetos grandes



Fragmentacion de Memoria

Fragmentacion Interna

Ocurre dentro de un bloque de memoria asignado que se esta desperdiciando

Fragmentacion Externa

Se produce cuando existen bloques de memoria libres dispersos, pero están aislados por bloques que aún están en uso



Gestion deterministica de Memoria

Determinismo implica que el tiempo necesario para asignar o liberar memoria debe ser constante.

Predictibilidad y Rendimiento Bruto

Un RTOS prioriza que las consecuencias de cualquier decisión de planificación sean analizables y anticipables

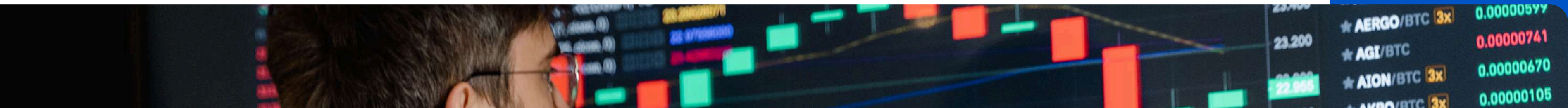
Es preferible un sistema que sea lento pero que garantice siempre cumplir sus plazos, a uno extremadamente rápido que, ocasionalmente y bajo carga, falle en una tarea crítica

Malloc y free

Muchos RTOS evitan el uso de malloc() y free(), porque sus algoritmos de busqueda depende de la fragmentación previa

No Determinismo

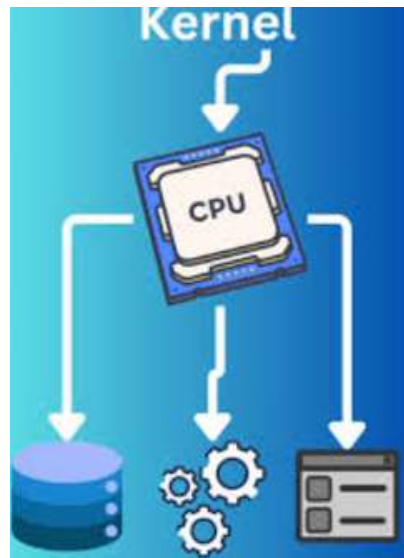
El algoritmo estándar de malloc() debe buscar un bloque contiguo de tamaño adecuado. A medida que la memoria se fragmenta, el tiempo de búsqueda fluctúa drásticamente.



Arquitectura de un RTOS

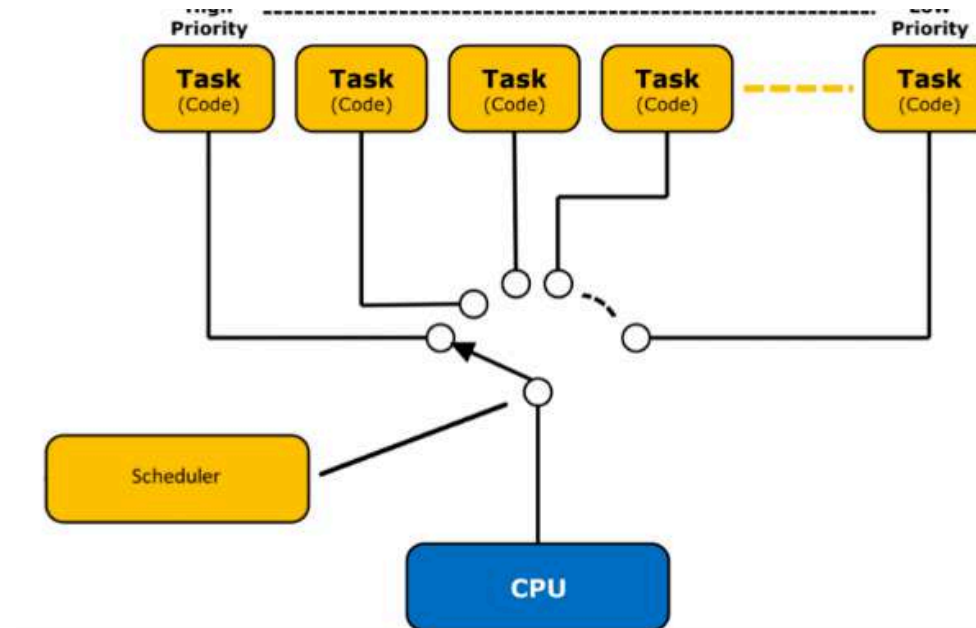
Kernel

El kernel es la parte más interna de un sistema operativo que tiene conexión directa con el hardware.



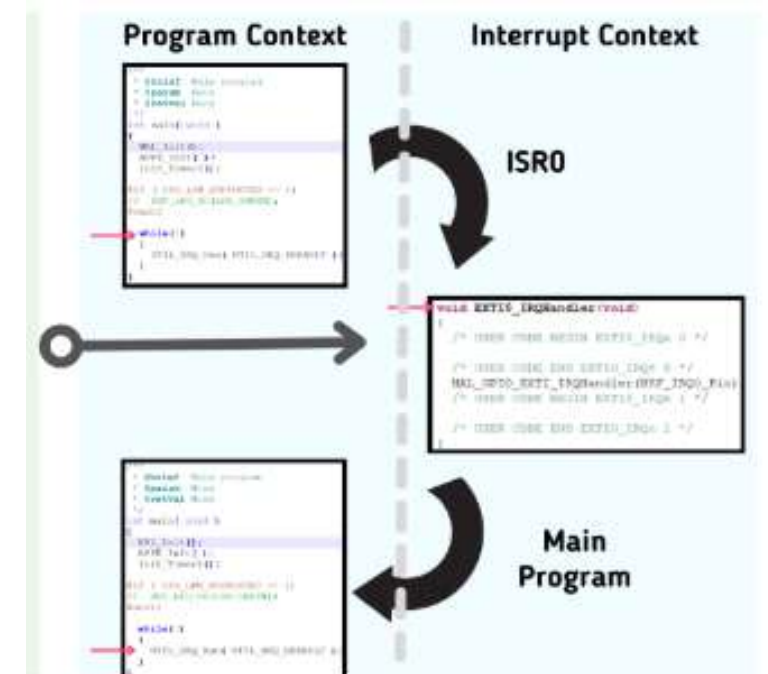
Scheduler(Planificador)

Es la parte del kernel encargada de determinar qué tarea se ejecuta a continuación basándose en algoritmos predefinidos



ISR(Rutina de Servicio de Interrupcion)

Es una subrutina especial a la que la CPU "salta" automáticamente cuando se reconoce un evento asíncrono externo (interrupción de hardware)



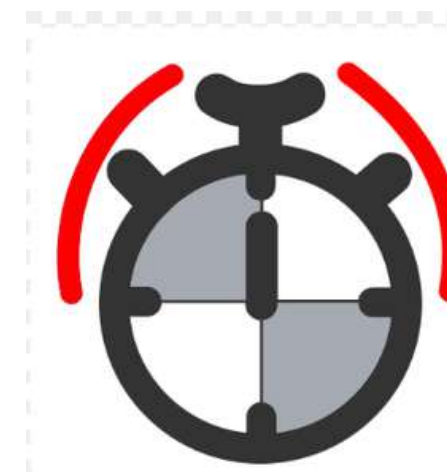
Drivers

Son módulos de software específicos diseñados para gestionar periféricos de entrada/salida (I/O)



Tick

Es una interrupción periódica especial generada por un temporizador de hardware (timer) que actúa como el "latido del corazón" del sistema



Tickless Systems(Sistemas sin Tick)

Es un modo de operación donde se suprimen los ticks periódicos del planificador en núcleos específicos dedicados a aplicaciones críticas

Microkernel vs Monolito

Esta arquitectura reduce los servicios esenciales del kernel al mínimo absoluto, típicamente limitándose a la planificación de tareas, la sincronización y la gestión básica de interrupciones

El microkernel prioriza la modularidad y la extensibilidad

Ejemplo Práctico: Un centro comercial. El equipo de mantenimiento central (el microkernel) solo se encarga de que los pasillos estén limpios y las puertas abiertas.



La arquitectura monolítica implementa el kernel como un programa único y extenso que integra la mayoría de los servicios del sistema operativo.

El monolito busca eficiencia extrema en sistemas cerrados

Ejemplo Práctico: : Una navaja suiza tiene todas las herramientas que están forjadas y soldadas a un mismo mango de metal sólido.



RTOS Populares y Casos Reales

RTOS Populares

FreeRTOS

Es un kernel de tiempo real ligero y de código abierto diseñado específicamente para microcontroladores pequeños con recursos limitados

VxWorks

Es un sistema operativo de tiempo real ampliamente utilizado en tecnologías sofisticadas de alta disponibilidad, como la industria aeroespacial

Zephyr

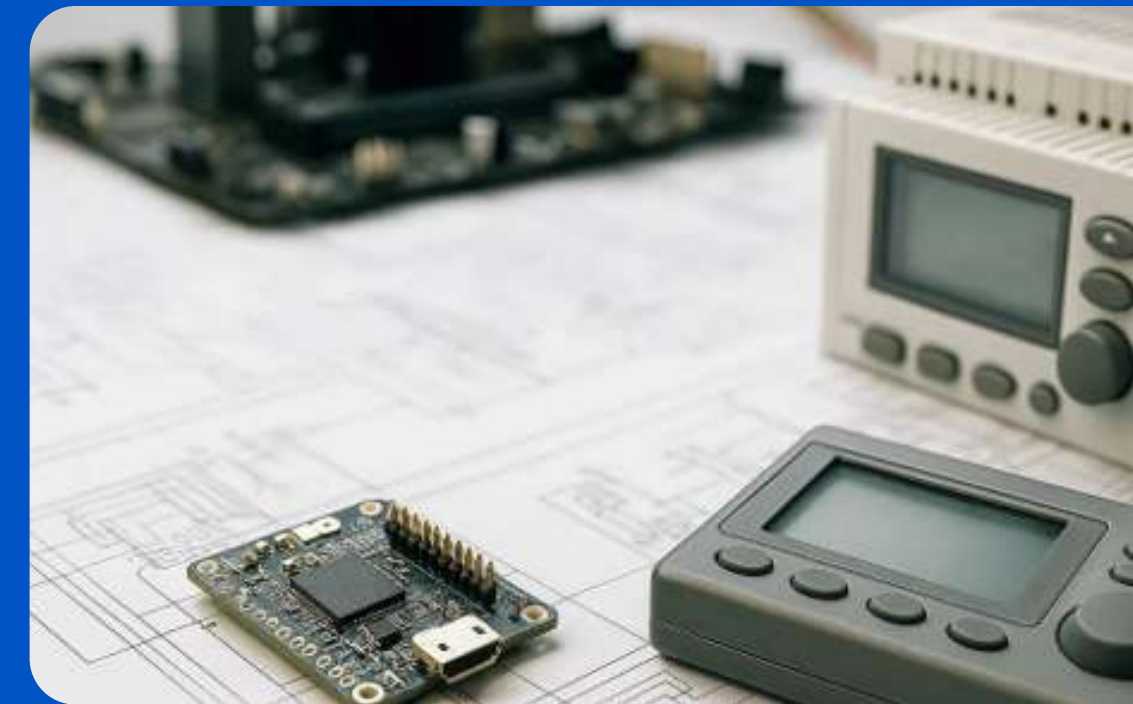
Se trata de un proyecto de RTOS de código abierto alojado por la Fundación Linux, diseñado para ser escalable y optimizado para la conectividad Bluetooth y Ethernet

QNX

Es un RTOS de arquitectura microkernel diseñado para aplicaciones de misión crítica que requieren alta escalabilidad y aislamiento de fallos

RTLinux

Se trata de una extensión de tiempo real para el sistema operativo Linux que inserta un ejecutivo de tiempo real pequeño entre el hardware y el kernel de Linux estándar



Casos Reales de Aplicación



ABS de vehículos

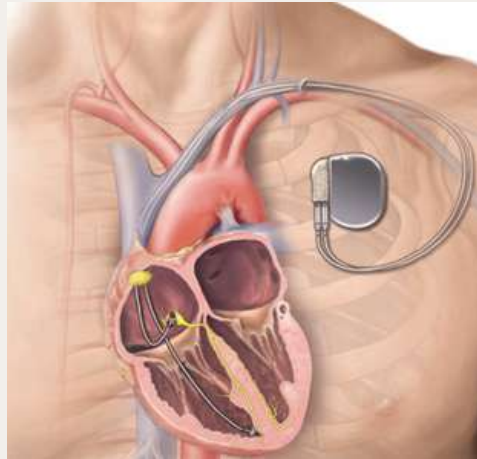
Aplicaciones de tiempo real crítico, los sensores de velocidad y los actuadores de los frenos deben interactuar en ciclos periódicos estrictos



Drones

Requieren RTOS especializados para ejecutar algoritmos de estabilidad de vuelo que deben gestionar múltiples entradas

Casos Reales de Aplicación



Marcapasos

Son dispositivos médicos implantables que utilizan RTOS de alta confiabilidad para monitorizar y regular los latidos cardíacos



Robots industriales

Los RTOS coordinan los movimientos complejos de manipuladores que deben interactuar con su entorno mediante sensores de fuerza y visión

THANK YOU

BIBLIOGRAFIA

- [1] M. Li, Z. Shang, Q. Hu, G. Yang, Y. Li, y F. Sun, "Analysis and Testing of Key Performance Indexes of Vxworks in Real-Time System," China Academy of Aerospace Systems Science and Engineering, Beijing, China, 2023.
- [2] M. Hubacz y B. Trybus, "Dual-Core PLC for Cooperating Projects with Software Implementation," Electronics, vol. 12, no. 23, p. 4730, dic. 2023.
- [3] S. Cuccagna, M. Femminella, G. Giusepponi, B. Guasticchi, L. Moriconi, y G. Reali, "Dynamic Deployment of Real Time Services in Edge Computing Systems," IEEE Open Journal of the Communications Society, vol. 7, p. 3658832, ene. 2026.
- [4] N. Silva, E. Alves, W. Júnior, A. Moares, N. Silva, y A. Balieiro, "FreeRTOS Application in a Real-Time Air Quality Monitoring Architecture for Smart Campus," Federal University of Southern and Southeastern Pará, Marabá, Brasil, 2024.
- [5] C. Antony Raj, "Innovations in Real-Time Operating Systems (RTOS) for Safety-Critical Embedded Systems," Journal of Computer Science and Technology Studies, vol. 7, no. 3, may. 2025
- [6] Wind River, "Learning Zephyr: Understanding its Implication & Features," Wind River Systems, Inc.
- [7] J. J. Labrosse, *MicroC/OS-II: The Real-Time Kernel*, 2da ed. Lawrence, KS: CMP Books, 2002.
- [8] H. Buczynski et al., "Resource Partitioning in Phoenix-RTOS for Critical and Noncritical Software for UAV Systems," Phoenix Systems Sp. z o.o., Varsovia, Polonia.
- [8] G. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4ta ed. Pisa, Italia: Springer Nature Switzerland AG, 2024
- [9] J. W. S. Liu, *Real-Time Systems*. Upper Saddle River, NJ: Prentice Hall, 2000.
- [10] Q. Li y C. Yao, *Real-Time Concepts for Embedded Systems*. San Francisco, CA: CMP Books, 2003.